



SovaScan

SYSTEM REFERENCE MANUAL

Technical Architecture & Workflow Manual

An executive guide detailing SovaScan's continuous code vulnerability analysis, local air-gapped deployment, and secure compliance integration model.

PREPARED FOR

Hackathon Judging Panel

PREPARED BY

Team Verstappen Paglus

Table of Contents

- 1. High-Level System Architecture 3
- 2. Frontend Dashboard Components 4
- 3. Backend & Database Layer 5
- 4. The 6-Stage Scan Engine 6
- 5. Alerting & Security Gateways 7
- 6. Live Demo Execution Checklists 8

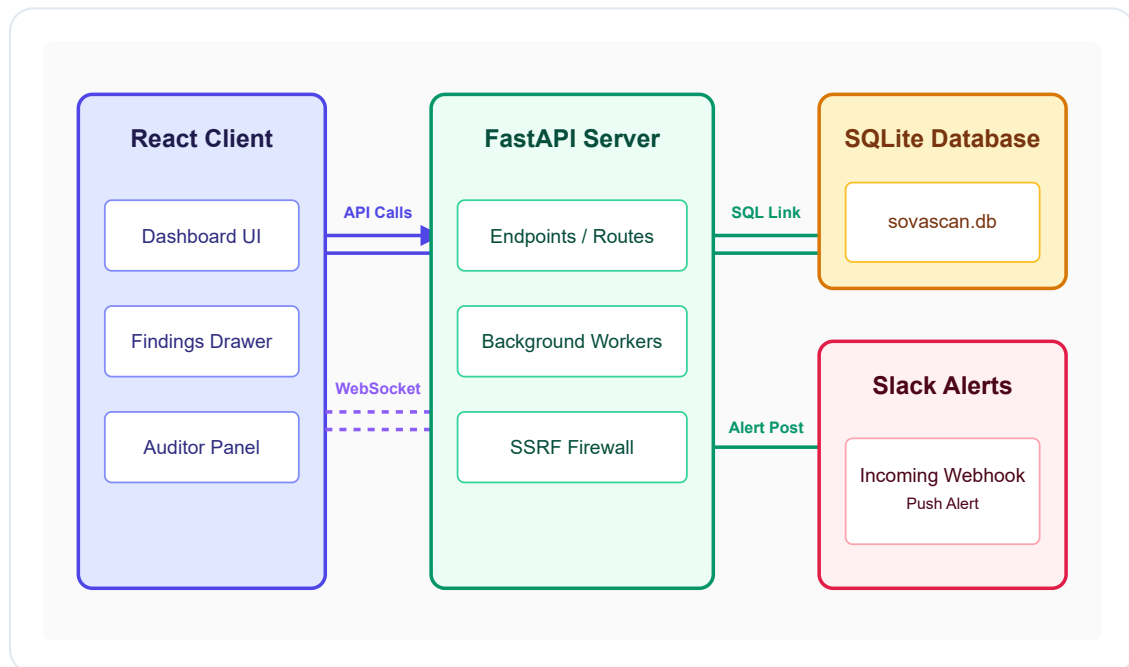
CHAPTER 1

High-Level System Architecture

SovaScan is designed specifically to operate inside air-gapped, zero-trust financial subnets. Rather than relying on external web servers or public cloud pipelines, it executes completely locally as an on-premises scanning sentinel.

The system is split into three main operational tiers: the React browser dashboard, the FastAPI web routing layer, and the background asynchronous scanning worker. Communications are managed via standard RESTful JSON calls and real-time WebSocket connection loops.

Operational Architecture Diagram



CHAPTER 2

Frontend Dashboard Components

The frontend application is constructed using React 18, compiled natively via the Vite bundler. It uses an Outfit-typography styling grid for high-end aesthetic layout presentation and is styled with a premium dark-mode interface.

Key Pages & UI Modules

- **Interactive Security Cockpit:** Displays dynamic charts (vulnerability distribution over time and severity ratio pie charts) built on top of Recharts libraries. It centers around a custom SVG HUD dial that represents the calculated overall compliance score.
- **Code Context Diff-Viewer:** When inspecting vulnerabilities, instead of basic output messages, the dashboard opens a slide-out drawer containing a 2-column diff pane. Unsafe lines are colored in crimson, and recommended replacements are rendered in green with a single-click action button.
- **Regulatory Framework Auditor:** Maps code findings directly into compliance indicators. It lists each control under RBI CSF (Section 3: Data Integrity, Section 7: Audit Trails) showing either a PASSED or FAILED status based on the database findings.

Aesthetic Polish Note: All card containers feature backdrop filter blurs (blur(20px)) with animated border flows, neon glow highlights on critical numbers, and subtle floating entry animations, making it immediately look like an enterprise-grade cyber command cockpit.

CHAPTER 3

Backend & Database Layer

The backend server is powered by FastAPI, leveraging Python's async core to process scanning operations in separate thread queues without blocking server request response lifecycles.

Security & API Integrity

The API handles secure credentials configuration storage and CLI executions safely:

```
# Hashing API keys securely for on-premises runner integration
import hashlib
import secrets

def generate_hashed_token(name: str):
    plaintext = f"ss_live_{secrets.token_hex(16)}"
    hashed = hashlib.sha256(plaintext.encode()).hexdigest()
    return plaintext, hashed
```

The system uses SQLAlchemy models to map python entities into a SQLite relational database file, `sovascan.db`. The primary tables include:

TABLE NAME	PRIMARY COLUMNS	RELATIONSHIP & DESCRIPTION
scans	id (UUID), target, status, started_at, completed_at	Has many findings; maps individual scanning events.
findings	id, scan_id, rule_id, severity, evidence, is_fixed	Belongs to a scan; registers specific security bugs.
api_keys	id, name, key_hash, is_active, created_at	Authenticates the SovaScan CLI commands.

TABLE NAME	PRIMARY COLUMNS	RELATIONSHIP & DESCRIPTION
audit_logs	id, timestamp, operator, action, justification	Immutable ledger log of user operations (like fixes).

CHAPTER 4

The 6-Stage Scan Engine

When a scan target path (e.g. `C:\Projects\bank-api`) is posted to SovaScan, the core engine initiates a 6-stage analysis worker thread. The progress of these stages is pushed immediately back to the browser console logs over WebSockets.

1. Workspace Discovery

Sweeps the directory to discover manifests, package lockfiles, environment configs, and code directories. If a Git URL is provided, it handles the secure checkout to a temporary sandbox directory.

2. Dependency & SBOM Resolution

Parses lockfiles (like `package.json` or `requirements.txt`) to build the Software Bill of Materials (SBOM) registry. It matches package names and versions against cached entries from the Open Source Vulnerabilities (OSV) database to catch known CVEs.

3. SAST (Static Application Security Testing)

Launches Bandit and Semgrep modules to audit code files for vulnerabilities (like command injection patterns or broken algorithms). It parses the resulting JSON outputs directly into SovaScan models.

4. Entropy & Regex Secret Auditing

Runs a regular expression scanner to catch obvious credentials (like AWS keys, GitHub tokens, Slack webhooks). In addition, it runs a Shannon Entropy algorithm to search for highly random string signatures (like base64 connection strings or unencrypted keys).

5. Context-Aware Threat Scorer

Adjusts the raw risk metrics of each vulnerability using local metadata context rules. A banking-related keyword in the file increases the severity weight by 1.5. Finding a bug inside test folders decreases the score, keeping alert focus strictly on production risks.

6. Regulatory Audit Mapping

Maps each finding to regulatory clauses. A hardcoded credential fails the **RBI CSF Data Integrity** control, updating compliance ratings automatically.

CHAPTER 5

Alerting & Security Gateways

When a scan completes, the results are formatted into a markdown card and sent to your Slack workspace. To protect banking networks, the webhook routing gateway incorporates an outbound SSRF (Server-Side Request Forgery) firewall.

SSRF Protection Workflow

SovaScan verifies every outgoing webhook request using a strict DNS-lookup filtering pipeline:

```
# Simplified python logic inside routes.py
import socket
from urllib.parse import urlparse

def secure_webhook_post(url: str):
    parsed = urlparse(url)
    if parsed.scheme != "https":
        raise ValueError("SSRF Guard: Only HTTPS scheme allowed")

    # Resolve host IP address
    ip_addr = socket.gethostbyname(parsed.hostname)

    # Check against private network blacklists
    if is_private_ip(ip_addr):
        raise ValueError("SSRF Guard: Blocked private IP target")

    # Proceed to post alert payload...
```

BLOCKED SUBNET RANGE	TYPE	ATTACK VECTOR PREVENTED
127.0.0.1 / ::1	Localhost Loopback	Preventing attackers from accessing server admin APIs.

BLOCKED SUBNET RANGE	TYPE	ATTACK VECTOR PREVENTED
169.254.169.254	Link-Local / Metadata	Preventing credentials theft from cloud metadata interfaces.
192.168.x.x / 10.x.x.x	Private Class A/C	Preventing scans of internal banking servers and subnets.

CHAPTER 6

Live Demo Execution Checklists

Use these structured scripts to ensure the judges see the technical rigor of the SovaScan platform during the live prototype demonstration.

1. The Setup

Place your Slack mobile app beside the laptop. Open the SovaScan UI on the main screen. Have the FastAPI terminal running in the background to show live logs.

2. Demonstration Walkthrough Steps

DEMO PHASE	ON-SCREEN ACTION	KEY CONCEPT TO HIGHLIGHT
1. Configuration	Navigate to Settings . Paste a malicious webhook target (e.g. <code>http://169.254.169.254</code>) and show the error block.	Explain Outbound SSRF Firewall defending local bank assets.
2. Live Scan	Go to New Scan . Paste the target directory path and trigger the scan. Watch the progress bar and real-time logs.	Explain the 6-stage asynchronous analysis engine and WebSockets.
3. Compliance Audit	Navigate to Compliance . Click on RBI CSF and show the gauge dials.	Explain how vulnerabilities map automatically to regulatory requirements .

DEMO PHASE	ON-SCREEN ACTION	KEY CONCEPT TO HIGHLIGHT
4. Auto-Remediation	Open Findings . Select a finding, click Auto-Fix , show the green diff, and click Apply Patch .	Show the one-click resolution and database audit log entries.
5. Push Alerting	Navigate to Settings , save your real Slack webhook, and click Test Webhook Alert .	Show the immediate push notification alert popping up on your phone.